

STORIFY



Technical Decisions & Architecture Document

1.0 | 2026

#	
1	System Architecture Overview —
2	Architecture Decision Records (ADRs) —
3	Frontend Architecture —
4	Backend Architecture — Backend
5	Database Architecture —
6	Multi-tenant Architecture —
7	Security Architecture —
8	Infrastructure & Deployment —
9	API Design Patterns — API
10	Error Handling & Logging —
11	Performance Considerations —
12	Testing Strategy —

1. System Architecture Overview



Storify is a Layered Monolith tenant. Backend Fastify application PostgreSQL schema isolation.

Cloudflare DNS
*.storify.com → Railway API
HTTPS
store1.storify.com admin.storify.com
Vercel
(React SPA) React + Vite + TailwindCSS + TanStack Query + Zustand
REST API (JSON)
Railway – Fastify API Server
Tenant Auth Feature Guard Middleware
Middleware
Route Modules
products | invoices | installments | stock | reports
BullMQ Queue Zod
Schemas Prisma ORM
PostgreSQL Redis (Upstash)
Cloudflare R2 master schema Sessions Images / PDFs
tenant_abc BullMQ Jobs Documents tenant_xyz Rate Limiting Signatures

1.1 Detailed Architecture

Presentation	React + Vite + TailwindCSS	SPA
API Gateway	Fastify + TypeScript	
Middleware Layer	Tenant + Auth + Feature Guards	
Business Logic	Route Handlers + Services	
Data Access	Prisma ORM	
Infrastructure	PostgreSQL + Redis + R2	Cache

2. Architecture Decision Records — ADRs

■■■ ■■■■■■■■■■ ■■■■■■■■ ■■ ■■■■■■■■ ■■■■■■■■

■■■ ■■■■■ ■■■■■ ■■■■■ ■■ ■■■■■■■■ ■■■■■■■■ ■■■■ ■■■■■■■■ ADR (Architecture Decision Record) ■■■■■■■■:
■■■■■■■■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■ ■■ ■■■■■■■■.

ADR-0
01

Fastify ■■■■■■ ■■ Express ■■ NestJS

Accepted

■■■■■■■■: ■■■■■■■■ ■■■■■■■■ backend ■■■■■ multi-tenant ■■ ■■■■■■■■ ■■■■■■■■■■ WebSockets
■■■■■■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■.

■■■■■■■■: ■■■■■■■■■■ Fastify + TypeScript ■■■ web framework ■■■■ backend.

■■■■■■■■: Fastify ■■■■■ ■■ Express ■■ 2x ■■ ■■■■ benchmarks. TypeScript first ■■■■■ native. Plugin system
■■■■■ ■■■■■ ■■■■■■■■ middleware stack ■■■■■. ■■■■■ WebSockets ■■■■■ ■■■■■■■■ ■■■■■■■■
sync ■■■■■■■■ ■■ ■■■■■■■■■■ ■■■■■■■■■■.

■■■■■■■■ ■■■■■■■■ ■■■■■■■■ multi-tenant ■■■■■■■■ event loop ■■■■■ ■■■■■■■■ ■■■■■■■■■■
■■: ■■■■■■■■■■. ■■■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■
■■■■■■■■■.

■■■■■■■■: NestJS: ■■■■■ ■■ ■■■■ structure ■■■■ overhead ■■■■ ■■■■■ ■■ ■■■■■ ■■■■■. Express: ■■■■■
■■■ ■■■■■ ■■■■■■■■ TypeScript native.

ADR-0
02

PostgreSQL Schemas ■■■■ Multi-tenant Isolation

Accepted

■■■■■■■■: ■■■■■■■■ SaaS ■■■■■ ■■■■■■■■ ■■■■■■■■■■. ■■ ■■■■■ ■■■■ ■■ ■■■■■ ■■■■■■■■■■ ■■■■■■■■
■■■■■■■■ ■■ ■■■■■■■■.

■■■■■■■■: ■■■ tenant ■■■■■ ■■■■ PostgreSQL Schema ■■■■■■■■ (tenant_{id}) ■■■■ ■■■■ ■■■■ database server.

■■■■■■■■: PostgreSQL Schemas ■■■■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■ ■■■■■■■■ ■■■■ ■■■■ connection
pool. ■■■■■ ■■■■■■■■ ■■ database ■■■■■■■■■■ ■■■■ ■■■■■. Prisma ■■■■■ multi-schema ■■■■■
■■■■■■■■. Migration ■■■■ tenant ■■■■■■■■.

■■■■■■■■ ■■■■■■■■ ■■■■■■■■ — SQL injection ■■ tenant ■■■■■ ■■ ■■■■■ ■■■■ ■■■■■■■■■■.
■■: ■■■■■■■■ Infrastructure ■■■■■■■■. ■■■■■■■■ ■■■■ backup ■■■■ ■■■■■ ■■■■■■■■■■.

■■■■■■■■: Database ■■■■■■■■ ■■■■ ■■■■■: ■■■■■ ■■■■■■■■ ■■■■ ■■■■■■■■ Infrastructure ■■■■■■■■■■.
Shared tables ■■ tenant_id: ■■■■■ ■■■■ ■■■■ ■■■■■■■■■■ ■■■■ ■■ bug ■■ ■■■■ queries.

ADR-0
03

Prisma ■■■ ORM

Accepted

■■■■■■■■: ■■■■■■■■ ■■■■■■■■ ORM ■■■■■ TypeScript ■■ multi-schema PostgreSQL ■■ ■■■■■■■■ ■■■■
migrations.

■■■■■■■■: ■■■■■■■■■■ Prisma ORM ■■■■■■■■■■ ■■ PostgreSQL ■■ ■■ schemas.

■■■■■■■■■■: TypeORM: ■■■■ type safety ■■■■ performance. Knex: raw SQL ■■■■ ■■■■ ■■■■ type safety. Drizzle: ■■■■ ■■■■ ■■■■ ecosystem ■■■■.

Accepted

Separate repos: types .
Nx/Turborepo: .

Accepted

Sessions Redis: Redis lookup request. Paseto:
JWT ecosystem .

Accepted

■■■■■: ■■■■■■ Zod ■■ validation ■■ request body ■query params ■env variables.

■■■■■■■:	TypeScript types ■Zod schemas ■■■■ ■■■■■■■■■■ ■■■■ Frontend ■Backend. Validation errors ■■■■■■ ■■■■■■■■. Parse don't validate — ■■■■ output ■■ Zod typed ■■■■■■■■■■. ■■■■ ■■■■ validation ■■■■■■■■.
■■■■■■■ ■■:	■■ ■■■■ ■■■■ invalid data ■■ ■■■■ ■■■■■■■■ ■■■■■■■■■■. ■■■■ env validation ■■■■ startup ■■■■ missing variables ■■■■■■. API errors ■■■■■■ ■■■■ client.
■■■■■■■:	Joi: ■■■■ ■■■■■■. Yup: ■■■■. Class-validator: ■■■■■■ decorators ■OOP.

ADR-007	BullMQ ■■■■ Background Jobs	Accepted
■■■■■■■:	■■■■■■■ ■■■■ PDF generation ■■■■■■■■ SMS ■■■■■■■■ Excel ■■■■■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■ API response.	
■■■■■■■:	■■■■■■■■■ BullMQ ■■■■ Redis ■■■■■■■■ background jobs ■■■■■■ blocking ■■■■ API.	
■■■■■■■:	PDF generation■ Excel export■ SMS notifications ■■■■ ■■■■ background. ■■■■ API ■■■■ ■■■■■■■■ ■■■■ job_id. ■■■■ user ■■■■■■■■ ■■■■■■■■■■ ■■■■ notification ■■ polling. BullMQ ■■■■■■ retries ■delay ■priority.	
■■■■■■■ ■■:	API latency ■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■. Reliability: ■■■■ jobs ■■■■■■■■ ■■■■ Redis ■■■■ ■■■■■■■■ ■■■■ server restart. Dashboard ■■■■■■■■ ■■■■ jobs (Bull Board).	
■■■■■■■:	Worker Threads: ■■■■ ■■■■ ■■■■■■. External queue services (SQS): ■■■■■■ reliability ■■■■ ■■■■■■■■ ■■■■■■.	

3. Frontend Architecture



3.1 Folder Structure

```
apps/web/src/
  api/
    client.ts ← axios instance
    interceptors
    JWT refresh
  auth.api.ts ← login, logout, refresh
  invoices.api.ts ← CRUD
  ...
  components/
    ui/ ← Button, Input, Modal, Table, Badge...
    layout/ ← Sidebar, Topbar, PageWrapper
  features/ ← InvoiceCard, ProductGrid, CartItem...
  pages/
    auth/ ← Login, Register
    dashboard/ ← Dashboard
    pos/ ←
    invoices/ ←
    +
    installments/ ←
    products/ ←
    stock/ ←
    customers/ ←
    suppliers/ ←
    expenses/ ←
    reports/ ←
    settings/ ←
  stores/
    auth.store.ts ← Zustand: user, tenant, permissions
    cart.store.ts ← Zustand: POS cart items
    ui.store.ts ← Zustand: sidebar, modals
  hooks/
  usePermission.ts ←
  useTenant.ts ←
  tenant
  useBarcode.ts ←
  types/
  index.ts ← shared types
@storify/shared
```

3.2 State Management Strategy

State		
Server State	TanStack Query	data API — caching + invalidation
Global Client State	Zustand	auth user cart UI state
Form State	React Hook Form + Zod	validation + submission
Local Component State	useState/useReducer	modal open/close toggle states
URL State	React Router v6	pagination filters current page

3.3 API Client Pattern

```
// api/client.ts
const apiClient = axios.create({
  baseURL: `https://${subdomain}.storify.com/api`,
});
// Request interceptor: Access Token
apiClient.interceptors.request.use((config) => {
  const token = authStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
// Response interceptor: Refresh Token
apiClient.interceptors.response.use(
  null,
  async (error) => {
    if (error.response?.status === 401) {
      const newToken = await refreshAccessToken();
      // HttpOnly Cookie
      error.config.headers.Authorization = `Bearer ${newToken}`;
      return apiClient(error.config);
    }
    return Promise.reject(error);
  }
);
```

4. Backend Architecture

■■■■■■■■ ■■■ Backend

4.1 Folder Structure

```
apps/api/src/ ■■■■ config/ ■ ■■■■ env.ts ← Zod env validation ■ ■■■■ database.ts ← Master DB +
getTenantDb() ■ ■■■■ redis.ts ← Redis connection ■■■■ modules/ ■ ■■■■ auth/ ■ ■ ■■■■
auth.routes.ts ■ ■ ■■■■ auth.service.ts ■ ■ ■■■■ auth.schema.ts ← Zod schemas ■ ■■■■ tenants/ ■
■ ■■■■ tenant.routes.ts ■ ■ ■■■■ tenant.service.ts ← provisionTenant() ■ ■ ■■■■ tenant.schema.ts
■ ■■■■ invoices/ ■ ■■■■ products/ ■ ■■■■ stock/ ■ ■■■■ installments/ ■ ■■■■ customers/ ■ ■■■■
suppliers/ ■ ■■■■ expenses/ ■ ■■■■ reports/ ■■■■ shared/ ■ ■■■■ middleware/ ■ ■ ■■■■
tenant.middleware.ts ← Tenant isolation ■ ■ ■■■■ auth.middleware.ts ← JWT verification ■ ■ ■■■■
feature.middleware.ts ← Plan feature guard ■ ■■■■ plugins/ ■ ■ ■■■■ jwt.plugin.ts ■ ■ ■■■■
cors.plugin.ts ■ ■ ■■■■ rateLimit.plugin.ts ■ ■■■■ utils/ ■ ■■■■ password.ts ← hash + verify ■
■■■■ fee.ts ← calculateFee() ■ ■■■■ decimal.ts ← financial calculations ■■■■ jobs/ ■ ■■■■
pdf.job.ts ← PDF generation ■ ■■■■ excel.job.ts ← Excel export ■ ■■■■ sms.job.ts ← SMS
notifications ■■■■ index.ts ← App entry point
```

4.2 Module Pattern — ■■ module ■■■■■■

```
// modules/invoices/invoice.routes.ts export async function invoiceRoutes(app: FastifyInstance) {
// ■■ route ■■■ ■■ middleware stack const preHandler = [ authenticate, // 1. ■■■■■■ ■■ JWT
tenantMiddleware, // 2. ■■■■■■ ■■■■ tenant requirePermission('invoices', 'create'), // 3. ■■■■
■■■■■■■■■ ]; app.post('/invoices', { preHandler }, createInvoiceHandler); app.get('/invoices', {
preHandler: [authenticate, tenantMiddleware] }, listInvoicesHandler); } //
modules/invoices/invoice.service.ts export async function createInvoice(db: PrismaClient, data:
CreateInvoiceDTO) { return db.$transaction(async (tx) => { // 1. ■■■■■■ ■■■■■■■■■■ const invoice
= await tx.invoice.create({ data: invoiceData }); // 2. ■■■■■■ ■■■■■■■■■■ ■■■ ■■■■■■ for (const
item of data.items) { await tx.stock.update({ where: { productId_branchId: { productId:
item.productId, branchId: data.branchId } }, data: { quantity: { decrement: item.quantity } }, });
await tx.stockMovement.create({ data: movementData }); } // 3. ■■■■■■ fee ■■■■■■■■ ■■■■■■■■ ■■
merchant ■■■■■■■■ if (data.feeBearer === 'merchant' && data.feeAmount > 0) { await
tx.paymentFeeExpense.create({ data: feeExpenseData }); } return invoice; }); }
```

[illegible]

```
-- Master DB (████ storify_master) -- █████ ███ █████ █████ ██████ ███ SELECT schema_name
FROM information_schema.schemata; -- public (master tables: plans, tenants, subscriptions) --
tenant_ahmed_store -- tenant_cairo_mall -- tenant_alex_fashion -- ... -- ███ tenant schema ██████
████ ███ █████ ██████████ -- tenant_ahmed_store.products -- tenant_ahmed_store.invoices --
tenant_cairo_mall.products ← █████████ █████████ █████████
```

```
// config/database.ts import { PrismaClient } from '@prisma/client'; // Master DB client - ■■■■
export const masterDb = new PrismaClient({ datasources: { db: { url: config.DATABASE_MASTER_URL }
}, }); // Cache ■■■■ tenant clients const tenantClients = new Map(); export function
getTenantDb(schemaName: string): PrismaClient { if (tenantClients.has(schemaName)) { return
tenantClients.get(schemaName)!; } // PostgreSQL schema ■■ connection string const url =
`${config.DATABASE_MASTER_URL}?schema=${schemaName}`; const client = new PrismaClient({
datasources: { db: { url } } }); tenantClients.set(schemaName, client); return client; } // ■■■■
connection pool ■■■■■■ ■■■■■■■■ ■■■ ■■■■ clients
```

■■■■■■■	■■■■■■■	■■■■■
DECIMAL(15,4) ■■■■■■	■■ ■■■■■■■■ ■■■■■■■■	■■■■ floating point errors ■■■ 0.1+0.2=0.30000000004
■■■ ■■■■■ ■■■ ■■■■■	fee, price, tax ■■ ■■■■■■■	■■ ■■■■ ■■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■ ■■ ■■■■■
■■ ■■■■■■■ ■■ ■■■ DB	■■■■■■■■ ■■■■■ ■■ ■■■ Service	■■■■■ ■■■ testing ■■■■ debugging
Decimal library ■■■■■■■■■	decimal.js ■■ ■■■ Backend	■■■ ■■■■■■■ ■■ ■■■ JavaScript
TIMESTAMP TZ ■■■■■■	■■ ■■■ timestamps ■■ timezone	■■■■ ■■■■■ ■■■■■■■■ ■■■■■■■■

Master DB Migration	Tenant Schema Migration	Prisma Migrate
Master DB Migration	Prisma Migrate —	
Tenant Schema Migration	Prisma Migrate	

□□□□□	□□□□	□□□□□
Rollback	■■■ ■■■■ bug ■■ migration	Prisma Migrate reset + restore
Zero-downtime Migration	■■ production	Additive changes + feature flags

6. Multi-tenant Architecture



6.1 Tenant Middleware Pipeline

```
// shared/middleware/tenant.middleware.ts export async function tenantMiddleware(request:
FastifyRequest, reply: FastifyReply) { // 1. subdomain const subdomain =
request.hostname.split('.')[0]; if (!['admin', 'www', 'api'].includes(subdomain)) return; // skip
system routes // 2. tenant Master DB cache const cacheKey =
`tenant:${subdomain}`; let tenant = await redis.get(cacheKey); if (!tenant) { tenant = await
masterDb.tenant.findUnique({ where: { slug: subdomain }, include: { plan: true }, }); if (tenant)
await redis.setex(cacheKey, 300, JSON.stringify(tenant)); // 5 min cache } if (!tenant ||
!tenant.isActive) { return reply.status(404).send({ error: 'tenant_not_found' }); } // 3.
if (isSubscriptionExpired(tenant)) { return reply.status(402).send({ error:
'subscription_expired', upgradeUrl: '/billing' }); } // 4. tenant context request
request.tenant = tenant; request.tenantDb = getTenantDb(tenant.schemaName); }
```

6.2 Feature Guard Middleware

```
// shared/middleware/feature.middleware.ts export function requireFeature(feature: keyof
PlanFeatures) { return async (request: FastifyRequest, reply: FastifyReply) => { const features =
request.tenant.plan.features as PlanFeatures; if (!features[feature]) { return
reply.status(403).send({ error: 'upgrade_required', feature, currentPlan:
request.tenant.plan.slug, upgradeUrl:
`https://${request.tenant.slug}.storify.com/billing/upgrade`, }); } } //
app.post('/installments', { preHandler: [authenticate, tenantMiddleware,
requireFeature('installments')] }, createInstallmentHandler );
```

6.3 Tenant Provisioning Flow

```
// modules/tenants/tenant.service.ts export async function provisionTenant(data: CreateTenantDTO):
Promise { const schemaName = `tenant_${data.slug.replace(/-/g, '_')}`; return
masterDb.$transaction(async (tx) => { // 1. tenant Master DB const tenant = await
tx.tenant.create({ data: { ...data, schemaName } }); // 2. PostgreSQL Schema await
masterDb.$executeRawUnsafe(`CREATE SCHEMA IF NOT EXISTS "${schemaName}"`); // 3.
Migrations await runTenantMigrations(schemaName); // 4. Seed const db =
getTenantDb(schemaName); await db.$transaction([ db.role.createMany({ data: DEFAULT_ROLES }),
db.branch.create({ data: { name: data.name, isActive: true } }), db.tenantSettings.create({ data:
DEFAULT_SETTINGS }), db.paymentMethod.createMany({ data: DEFAULT_PAYMENT_METHODS }),
db.taxRate.createMany({ data: DEFAULT_TAX_RATES }), db.expenseCategory.createMany({ data:
DEFAULT_EXPENSE_CATEGORIES }), db.currency.create({ data: { code: 'EGP', name: 'EGP',
rateToBase: 1, isBase: true } }), ]); return tenant; }); }
```

7. Security Architecture



7.1 Security Layers

Layer	Implementation	Threats
Transport	HTTPS / TLS 1.3 (Cloudflare)	Man-in-the-middle
Authentication	JWT + HttpOnly Cookie Refresh	Session hijacking, XSS
Authorization	RBAC + Permission Guards	Privilege escalation
Tenant Isolation	PostgreSQL Schema per tenant	Data leakage between tenants
Input Validation	Zod schemas <code>z.string().min(1).max(100)</code> input	SQL Injection, XSS, Type errors
Rate Limiting	@fastify/rate-limit <code>1000</code> Redis	Brute force, DDoS
Security Headers	@fastify/helmet	XSS, Clickjacking, MIME sniffing
CORS	<code>Access-Control-Allow-Origin: https://subdomain</code> subdomain	Cross-origin requests
Password Security	SHA-256 + random salt	Rainbow table attacks
Financial Precision	DECIMAL(15,4) + decimal.js	Floating point manipulation

7.2 Permission System

```
// shared/middleware/auth.middleware.ts export function requirePermission(resource: string,
action: string) { return async (request: FastifyRequest, reply: FastifyReply) => { const user =
request.user as JWTpayload; const permissions = user.permissions as Record; // Super Admin
if (user.roleSlug === 'super_admin') return; if
(!permissions[resource]?.includes(action)) { return reply.status(403).send({ error: 'forbidden',
required: `${resource}:${action}`, userRole: user.roleSlug, }); } } } // JWT Payload Structure
interface JWTpayload { userId: string; tenantId: string; schemaName: string; roleSlug: string;
branchId: string; permissions: Record; // DB login iat: number; exp: number; }
```

8. Infrastructure & Deployment



8.1 Railway Configuration

```
# railway.toml [build] builder = "nixpacks" [deploy] startCommand = "node dist/index.js"
healthcheckPath = "/health" healthcheckTimeout = 30 restartPolicyType = "on_failure"
[environments.production] NODE_ENV = "production" # Environment Variables ■■ Railway Dashboard: #
DATABASE_MASTER_URL → Railway PostgreSQL # REDIS_URL → Upstash Redis # JWT_ACCESS_SECRET →
generated secret # JWT_REFRESH_SECRET → generated secret # R2_* → Cloudflare R2 credentials
```

8.2 CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/deploy.yml name: Deploy to Production on: push: branches: [main] jobs: test:
runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: pnpm/action-setup@v3 - run: pnpm
install - run: pnpm type-check # TypeScript check - run: pnpm lint # ESLint - run: pnpm test #
Vitest deploy-api: needs: test runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - run:
railway up --service api # Railway CLI deploy deploy-web: needs: test runs-on: ubuntu-latest
steps: - uses: actions/checkout@v4 - run: vercel --prod # Vercel deploy
```

8.3 ■■■■■■ ■■■ Infrastructure

■■■■■	■■■■■	■■■■■	■■■■■/■■■	■■■■
API + DB	Railway	Pro	~\$20	8GB RAM■ PostgreSQL
Frontend	Vercel	Hobby	■■■■■	100GB bandwidth
Redis	Upstash	Pay-as-you-go	■■■■■→\$0.2	10K commands/day ■■■■■■
Storage	Cloudflare R2	Pay-as-you-go	■■■■■→\$0.015/GB	10GB ■■■■■■
DNS + CDN	Cloudflare	Free	■■■■■	Wildcard SSL ■■■■■■
Email	Resend	Free	■■■■■	3000 email/■■■■
■■■■■■■■■	—	—	~\$20	■■■■■ ■■■■ 20-50 ■■■■■

9. API Design Patterns

■■■■■ ■■■■■ ■■■ API

9.1 URL Structure

■■■ Endpoint	■■■ Method	■■■■■
/api/auth/login	POST	■■■■■ ■■■■■■
/api/auth/refresh	POST	■■■■■ ■■■ Access Token
/api/products	GET	■■■■■ ■■■■■■■■■ (■■■ pagination + filters)
/api/products/:id	GET	■■■■■ ■■■■
/api/products	POST	■■■■■ ■■■■ ■■■■
/api/products/:id	PATCH	■■■■■ ■■■■
/api/invoices	POST	■■■■■ ■■■■■■ + ■■■■■ ■■■■■ + fee expense
/api/installments/:id/approve	PATCH	■■■■■ ■■■■ ■■■ ■■■ ■■■■■
/api/installments/:id/payment	POST	■■■■■ ■■■■ ■■■
/api/reports/dashboard	GET	■■■■■ ■■■ Dashboard
/api/reports/sales	GET	■■■■■ ■■■■■■■■ (■■■ date filters)
/api/reports/fees	GET	■■■■■ ■■■■ ■■■ ■■■■■

9.2 Response Format Standard

```
// Success Response { "success": true, "data": { ... }, "meta": { // ■■■ lists ■■■ "total": 234,
"page": 1, "limit": 20, "pages": 12 } } // Error Response { "success": false, "error": { "code":
"validation_error", // machine-readable "message": "■■■■■■■■ ■■■ ■■■■■", "details": [ // Zod
validation errors { "field": "email", "message": "■■■■■■■■ ■■■ ■■■■" } ] } } // Error Codes //
validation_error → 400 // unauthorized → 401 // forbidden → 403 // not_found → 404 //
subscription_expired → 402 // upgrade_required → 403 (feature not in plan) // internal_error →
500
```

10. Error Handling & Logging



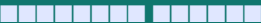
10.1 Global Error Handler

```
// shared/plugins/errorHandler.ts app.setErrorHandler((error, request, reply) => { // Zod Validation Error if (error instanceof ZodError) { return reply.status(400).send({ success: false, error: { code: 'validation_error', message: 'Validation failed: ' + error.flatten().fieldErrors.join(', '), details: error.flatten().fieldErrors, }, }); } // Prisma Known Error (Unique constraint) if (error instanceof Prisma.PrismaClientKnownRequestError) { if (error.code === 'P2002') { // Unique constraint return reply.status(409).send({ success: false, error: { code: 'duplicate_entry', message: 'Duplicate entry: ' + error.message, }, }); } } // Log unexpected errors app.log.error({ err: error, requestId: request.id, tenantId: request.tenant?.id }); return reply.status(500).send({ success: false, error: { code: 'internal_error', message: 'Internal server error', }, }); });
```

10.2 Logging Strategy

Level	Message	Details
info	Application started successfully	POST /invoices — 200 — 45ms — tenant: ahmed
warn	Rate limit reached for IP: 192.168.1.1	Rate limit hit — IP: x.x.x.x
error	Database connection failed	DB connection failed — stack trace
debug	Query executed successfully	Query: SELECT * FROM products — 12ms

11. Performance Considerations



11.1 Database Query Optimization

████████████████████	████████	██████
Connection Pooling	Prisma ██████ █████ pool ██████████████	██ overhead █████ connections █████
Tenant DB Caching	Map	████ █████ █████ client ██ █ request
Tenant Info Caching	Redis cache ██ 5 █████	████ Master DB query ██ █ request
Select specific fields	{ select: { id, name, price } }	██████ data transfer ██ DB
Pagination	cursor-based ██ lists █████	████ █████ █ █████ █████████
Indexes	████ barcode, email, subdomain	████ █████ █████ lookups

11.2 Caching Strategy

```
// Tenant info – Redis 5 minutes redis.setex(`tenant:${subdomain}`, 300, JSON.stringify(tenant));
// Plan features – Redis 10 minutes redis.setex(`plan:${planId}:features`, 600,
JSON.stringify(features)); // Dashboard stats – Redis 2 minutes (fresh enough, fast enough)
redis.setex(`dashboard:${tenantId}:${branchId}`, 120, JSON.stringify(stats)); // Product by
barcode – Redis 30 minutes redis.setex(`product:${tenantId}:barcode:${barcode}`, 1800,
JSON.stringify(product)); // Cache invalidation – █████ ████████████████
redis.del(`product:${tenantId}:barcode:${barcode}`); // █████ ████████████████
```

[illegible]

12.1 Testing Pyramid

Unit Tests	Integration Tests	E2E Tests	Load Tests
Unit Tests	Integration Tests	E2E Tests	Load Tests
Vitest	Vitest + Prisma Test	Playwright	k6
calculateFee(), hashPassword(), Zod schemas	API endpoints ■■ DB ■■■■■■■	POS flow■ login■ installment approval	multi-tenant scenarios ■■■ ■■■
■■■■■	■■■■■	■■■■■■■	■■■■■■■

12.2 Critical Test Cases

Test	
calculateFee: percentage + fixed + both + none	
tenant isolation: tenant A tenant B	security requirement
installment approval flow: pending → active	business flow
invoice creation: stock update + fee expense atomically	Transaction integrity
JWT refresh: 15	User experience
feature guard: plan downgrade	SaaS billing correctness
decimal precision: 0.1 + 0.2 = 0.3 (0.30000000004)	Financial accuracy